

CSV Dataset: flights.csv

Create the file in Colab:

```
%writefile flights.csv
booking_id,passenger_name,from_city,to_city,airline,ticket_price,travel_status
1001,Aarav Mehta,Hyderabad,Delhi,IndiGo,6500,Economy,Confirmed
1002,Sana Khan,Bangalore,Mumbai,Vistara,8200,Economy,Confirmed
1003,John Mathew,Chennai,Delhi,Air India,12000,Business,Confirmed
1004,Ayesha Begum,Hyderabad,Dubai,Emirates,28000,Economy,Confirmed
1005,Vikram Rao,Mumbai,Singapore,Singapore Airlines,35000,Business,Pending
1006,Divya Sharma,Delhi,Hyderabad,IndiGo,5900,Economy,Cancelled
1007,Imran Ali,Pune,Bangalore,Akasa Air,4800,Economy,Confirmed
1008,Meera Nair,Kochi,Dubai,Emirates,26000,Economy,Confirmed
1009,Rohan Das,Kolkata,Delhi,Air India,7400,Economy,Pending
1010,Nisha Reddy,Bangalore,London,British Airways,62000,Business,Confirmed
```

Read CSV:

```
flights_df = spark.read.csv(
    "flights.csv",
    header=True,
    inferSchema=True
)

flights_df.show()
flights_df.printSchema()
```

CSV Exercises

1. Display all flight bookings.
2. Display passenger_name, from_city and to_city.
3. Display only confirmed bookings.
4. Display cancelled bookings.
5. Display pending bookings.
6. Display bookings where ticket_price > 20000.

7. Display international flights to Dubai, Singapore or London.
8. Count total bookings.
9. Count bookings by airline.
10. Count bookings by status.
11. Find total revenue from confirmed bookings.
12. Find average ticket price by airline.
13. Find highest ticket price.
14. Find lowest ticket price.
15. Sort bookings by ticket_price descending.
16. Create tax column = ticket_price * 0.05.
17. Create final_price column = ticket_price + tax.
18. Create price_category:
 ticket_price >= 30000 -> Premium
 ticket_price >= 10000 -> Standard
 Else -> Budget
19. Count bookings by price_category.
20. Save confirmed bookings as confirmed_flights.parquet.

JSON Dataset: hotels.json

Create the file in Colab:

```
%%writefile hotels.json
[
  {
    "hotel_id": 201,
    "hotel_name": "Pearl Grand",
    "city": "Hyderabad",
    "category": "Business",
    "rating": 4.4,
    "rooms_available": 25,
    "price_per_night": 4500,
    "amenities": ["wifi", "breakfast", "gym"],
    "contact": {
      "phone": "9876500011",
      "email": "pearlgrand@mail.com"
    }
  },
  {
    "hotel_id": 202,
    "hotel_name": "Marina Bay Stay",
```

```
    "city": "Dubai",
    "category": "Luxury",
    "rating": 4.8,
    "rooms_available": 12,
    "price_per_night": 18000,
    "amenities": ["wifi", "pool", "spa", "sea_view"],
    "contact": {
      "phone": "9876500012",
      "email": "marinabay@mail.com"
    }
  },
  {
    "hotel_id": 203,
    "hotel_name": "Budget Inn",
    "city": "Delhi",
    "category": "Budget",
    "rating": 3.9,
    "rooms_available": 40,
    "price_per_night": 2200,
    "amenities": ["wifi"],
    "contact": {
      "phone": null,
      "email": "budgetinn@mail.com"
    }
  },
  {
    "hotel_id": 204,
    "hotel_name": "Hill View Resort",
    "city": "Kochi",
    "category": "Resort",
    "rating": 4.5,
    "rooms_available": 18,
    "price_per_night": 7500,
    "amenities": ["wifi", "breakfast", "pool"],
    "contact": {
      "phone": "9876500014",
      "email": null
    }
  },
  {
    "hotel_id": 205,
    "hotel_name": "Skyline Suites",
    "city": "London",
    "category": "Luxury",
```

```

    "rating": 4.7,
    "rooms_available": 8,
    "price_per_night": 22000,
    "amenities": ["wifi", "breakfast", "spa"],
    "contact": {
        "phone": "9876500015",
        "email": "skyline@mail.com"
    }
}
]

```

Read JSON:

```

hotels_df = spark.read.option(
    "multiline",
    "true"
).json("hotels.json")

hotels_df.show(truncate=False)
hotels_df.printSchema()

```

JSON Exercises

1. Display all hotel records.
2. Display hotel_name, city and rating.
3. Display luxury hotels.
4. Display hotels with rating greater than 4.5.
5. Display hotels where rooms_available > 15.
6. Display hotels where price_per_night > 10000.
7. Display hotels from Dubai or London.
8. Display hotels where contact.phone is null.
9. Display hotels where contact.email is null.
10. Display hotel_name with contact.phone and contact.email.
11. Display hotels having amenity wifi.
12. Display hotels having amenity spa.
13. Count hotels by city.
14. Count hotels by category.
15. Find average rating by category.
16. Find average price_per_night by city.
17. Find highest price_per_night.

18. Create `total_potential_revenue = rooms_available * price_per_night`.
19. Sort hotels by rating descending.
20. Flatten JSON and save as `hotels_flattened.parquet`.